МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность ПО-13

Выполнил
Бондарчук А.Ю,
студент группы
ПО-13

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«_____k ___2017 г.

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python.

Задание 1.

2) Равносторонний треугольник, заданный длинами сторон – Предусмотреть возможность определения площади и периметра, а также логический метод, определяющий существует или такой треугольник. Конструктор должен позволять создавать объекты с начальной инициализацией. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Код программы

```
import math
    import sys

    class Triangular:
        """класс_треугольник"""

        def __init__(self, a = 3, b = 3, c = 3) :
            self.a = a
            self.b = b
            self.c = c

        def print(self) :
            print("a",self.a)
            print("b",self.b)
            print("c",self.c)

        def perimeter(self) :
            return self.a + self.b + self.c

        def area(self) :
            p = self.perimeter()
            return math.sqrt(p * (p - self.a) * (p - self.b) * (p - self.c))

        def is_exists(self) :
            return self.a + self.b >= self.c and self.a + self.c >= self.b and
    self.c + self.b >= self.a

        def __eq__(self, other) :
            return self.a == other.a and self.b == other.b and self.c ==
    other.c

    if __name__ == '__main__':
        triangular_1 = Triangular(3, 3, 3)
        triangular_2 = Triangular(1, 2, 3)
        triangular_3 = Triangular()

        #"""данные"""
        print("triangular_1 = ")
        triangular_1.print()
        print("triangular_2 = ")
        triangular_2.print()
        print("triangular_3 = ")
        triangular_3.print()

        #"""проверка"""
        if triangular_1.is_exists() :
            print("Треугольник triangular_1 существует")
        else :
            print("Треугольник triangular_1 несуществует")
            sys.exit()
        if triangular_2.is_exists() :
            print("Треугольник triangular_2 существует")
        else :
            print("Треугольник triangular_2 несуществует")
```

```
        sys.exit()
    if triangular_3.is_exists():
        print("Треугольник triangular_3 существует")
    else:
        print("Треугольник triangular_3 несуществует")
        sys.exit()

    #сравнение
    if triangular_1 == triangular_2 :
        print("Треугольник triangular_2 = triangular_1")
    else:
        print("Треугольник triangular_2 != triangular_1")
    if triangular_1 == triangular_3 :
        print("Треугольник triangular_2 = triangular_1")
    else:
        print("Треугольник triangular_2 != triangular_1")
```

Спецификация вывода

triangular_1: a <значение> b <значение> c <значение>
triangular_2: a <значение> b <значение> c <значение>
triangular_3: a <значение> b <значение> c <значение>
existence: <существует / не существует>
compare_1: triangular_2 != triangular_1
compare_2: triangular_2 = triangular_1

Пример

triangular_1 =
a 3
b 3
c 3
triangular_2 =
a 1
b 2
c 3
triangular_3 =
a 3
b 3
c 3
Треугольник triangular_1 существует
Треугольник triangular_2 существует
Треугольник triangular_3 существует
Треугольник triangular_2 != triangular_1
Треугольник triangular_2 = triangular_1

Рисунки с результатами работы программы

```
triangular_1 =
a 3
b 3
c 3
triangular_2 =
a 1
b 2
c 3
triangular_3 =
a 3
b 3
c 3
Треугольник triangular_1 существует
Треугольник triangular_2 существует
Треугольник triangular_3 существует
Треугольник triangular_2 != triangular_1
Треугольник triangular_2 = triangular_1
```

Задание 2.

2) Система Платежи. Клиент имеет Счет в банке и Кредитную Карту (КК).

Клиент может оплатить Заказ, сделать платеж на другой Счет, заблокировать

КК и аннулировать Счет. Администратор может заблокировать КК за превышение кредита.

Код программы

```
class Client:
    def __init__(self, name, credit_card):
        self.name = name
        self.credit_card = credit_card

    def print(self):
        print(self.name)
        print(self.credit_card)

    def pay_for_order(self, order):
        self.credit_card.pay(order)

    def transfer_money(self, account, amount):
        self.credit_card.transfer(account, amount)

    def block_kk(self, credit_card):
        credit_card.block()

    def null_account(self):
        self.credit_card.null_account()


class Card:
    def __init__(self, account, number, status):
        self.account = account
        self.number = number
        self.status = status
```

```
    def get_balance(self):
        return self.account.get_balance()

    def pay(self, order):
        if order.cost <= self.get_balance() and order.status != "Paid" and
self.status != "Blocked":
            self.account.pay(order.get_cost())
            order.set_status("Paid")
            print("Оплата прошла успешно")
        else:
            print("Операция не прошла!")

    def transfer(self, account, amount):
        if amount <= self.get_balance() and self.status != "Blocked":
            self.account.pay(amount)
            account.get_money(amount)
            print("Перевод выполнен успешно")
        else:
            print("Операция не прошла!")

    def block(self):
        self.status = "Blocked"
        print("Карта заблокирована")

    def null_account(self):
        self.account.null_account()

    def transfer_money(self, account, amount):
        if amount <= self.get_balance() and self.status != "Blocked":
            self.account.pay(amount)
            account.get_money(amount)
        else:
            print("Операция не прошла!")


class Order:
    def __init__(self, cost, status="Waiting"):
        self.cost = cost
        self.status = status

    def print(self):
        print(self.status)

    def set_status(self, status):
        self.status = status

    def get_cost(self):
        return self.cost

class BankAccount:
    def __init__(self, owner, credit_card, balance):
        self.owner = owner
        self.credit_card = credit_card
        self.balance = balance

    def get_balance(self):
        return self.balance

    def set_balance(self, balance):
        self.balance = balance

    def pay(self, amount):
        self.set_balance(self.balance - amount)
        if self.balance < 0:
            self.credit_card.block()
            print("Карта заблокирована из-за отрицательного баланса")

    def get_money(self, amount):
```

```
        self.set_balance(self.balance + amount)

    def null_account(self):
        self.balance = 0
        print("Счет аннулирован")


class Administrator(Client):
    def __init__(self,name,credit_card=None):
        Client.__init__(self,name,credit_card)
    def block_kk(self, credit_card):
        credit_card.block()
        print(f"Администратор {self.name} заблокировал карту")


if __name__ == '__main__':
    bank_account = BankAccount("Иван Петров", None, 5000)
    var_credit_card = Card(bank_account, "4276-1234-5678-9012", "Active")
    bank_account.credit_card = var_credit_card

    client = Client("Иван Петров", var_credit_card)
    admin = Administrator("Петр Сидоров")

    print(f"Клиент: {client.name}")
    print(f"Номер карты: {var_credit_card.number}")
    print(f"Статус карты: {var_credit_card.status}")
    print(f"Баланс счета: {bank_account.get_balance()}")
    print(f"Баланс карты: {var_credit_card.get_balance()}")

    order1 = Order(2000, "Waiting")
    order2 = Order(4000, "Waiting")
    order3 = Order(1500, "Waiting")

    print(f"Заказ 1: сумма {order1.cost}, статус {order1.status}")
    print(f"Заказ 2: сумма {order2.cost}, статус {order2.status}")
    print(f"Заказ 3: сумма {order3.cost}, статус {order3.status}")

    client.pay_for_order(order1)
    print(f"Статус заказа: {order1.status}")
    print(f"Баланс после оплаты: {bank_account.get_balance()}")

    client.pay_for_order(order2)
    print(f"Статус заказа: {order2.status}")
    print(f"Баланс после оплаты: {bank_account.get_balance()}")
    print(f"Статус карты: {var_credit_card.status}")


    other_account = BankAccount("Петр Иванов", None, 1000)
    print(f"Баланс счета {other_account.owner} до перевода:
{other_account.get_balance()}")

    client.transfer_money(other_account, 500)
    print(f"Баланс счета {client.name} после перевода: {bank_account.get_balance()}")
    print(f"Баланс счета {other_account.owner} после перевода:
{other_account.get_balance()}")

    client.block_kk(var_credit_card)
    print(f"Статус карты: {var_credit_card.status}")

    client.pay_for_order(order3)
    print(f"Статус заказа: {order3.status}")
    print(f"Баланс не изменился: {bank_account.get_balance()}")

    new_account = BankAccount("Сергей", None, 3000)
    new_card = Card(new_account, "5555-6666", "Active")
    new_account.credit_card = new_card
```

```
print("Новая карта, статус: ",new_card.status)

admin.block_kk(new_card)
print("Статус карты после блокировки: ",new_card.status)

print(f"Баланс до аннулирования: {bank_account.get_balance()}")
client.null_account()
print(f"Баланс после аннулирования: {bank_account.get_balance()}")
```

Спецификация вывода

client: <имя клиента>
card_number: <номер карты>
card_status: <статус карты>
account_balance: <баланс счета>
card_balance: <баланс карты>
orders:
order_1: amount <сумма>, status <статус>
order_2: amount <сумма>, status <статус>
order_3: amount <сумма>, status <статус>
payment_result: <результат оплаты>
order_status: <статус заказа>
balance_after_payment: <баланс>
transfer:
receiver_balance_before: <баланс>
transfer_result: <результат>
sender_balance_after: <баланс>
receiver_balance_after: <баланс>
card_block:
card_status_after_block: <статус>
admin_action:
admin_name: <имя администратора>
card_status_after_admin_block: <статус>
account_annulment:
balance_before: <баланс>
annulment_status: <результат>
balance_after: <баланс>

Пример

Клиент: Иван Петров
Номер карты: 4276-1234-5678-9012
Статус карты: Active
Баланс счета: 5000
Баланс карты: 5000
Заказ 1: сумма 2000, статус Waiting
Заказ 2: сумма 4000, статус Waiting
Заказ 3: сумма 1500, статус Waiting
Оплата прошла успешно
Статус заказа: Paid
Баланс после оплаты: 3000
Операция не прошла!
Статус заказа: Waiting

Баланс после оплаты: 3000
Статус карты: Active
Баланс счета Петр Иванов до перевода: 1000
Перевод выполнен успешно
Баланс счета Иван Петров после перевода: 2500
Баланс счета Петр Иванов после перевода: 1500
Карта заблокирована
Статус карты: Blocked
Операция не прошла!
Статус заказа: Waiting
Баланс не изменился: 2500
Новая карта, статус: Active
Карта заблокирована
Администратор Петр Сидоров заблокировал карту
Статус карты после блокировки: Blocked
Баланс до аннулирования: 2500
Счет аннулирован
Баланс после аннулирования: 0

Рисунки с результатами работы программы

```
Клиент: Иван Петров
Номер карты: 4276-1234-5678-9012
Статус карты: Active
Баланс счета: 5000
Баланс карты: 5000
Заказ 1: сумма 2000, статус Waiting
Заказ 2: сумма 4000, статус Waiting
Заказ 3: сумма 1500, статус Waiting
Оплата прошла успешно
Статус заказа: Paid
Баланс после оплаты: 3000
Операция не прошла!
Статус заказа: Waiting
Баланс после оплаты: 3000
Статус карты: Active
Баланс счета Петр Иванов до перевода: 1000
Перевод выполнен успешно
Баланс счета Иван Петров после перевода: 2500
Баланс счета Петр Иванов после перевода: 1500
Карта заблокирована
Статус карты: Blocked
Операция не прошла!
Статус заказа: Waiting
Баланс не изменился: 2500
Новая карта, статус:  Active
Карта заблокирована
Администратор Петр Сидоров заблокировал карту
Статус карты после блокировки:  Blocked
Баланс до аннулирования: 2500
Счет аннулирован
Баланс после аннулирования: 0
```

Вывод: закрепил навыки объектно-ориентированного программирования на языке Python.